1. FCFS (First Come First Serve)

FCFS is the simplest CPU scheduling algorithm where processes are executed in the order they arrive. It follows a queue structure (FIFO). The process that arrives first gets executed first without interruption. It is easy to implement but can cause long waiting times for shorter processes.

FIFO.cpp > main()

```cpp

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int at[n], bt[n], wt[n], tat[n];

    for(int i = 0; i < n; i++) {
        cout << "Enter Arrival Time and Burst Time for P" << i+1 << ": ";
        cin >> at[i] >> bt[i];
    }

    // Sort by Arrival Time
    for(int i = 0; i < n; i++) {
        for(int j = i+1; j < n; j++) {
            if(at[i] > at[j]) {
                swap(at[i], at[j]);
                swap(bt[i], bt[j]);
            }
        }
    }

    wt[0] = 0;
    for(int i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    for(int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    cout << "\nProcess\tAT\tBT\tWT\tTAT\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << i+1 << "\t" << at[i] << "\t" << bt[i] << "\t" << wt[i] << "\t" << tat[i] << endl;
    }
}
```

```
PS C:\Users\VIKRAMJEET\Desktop\OS Assignment>
Enter number of processes: 5
Enter Arrival Time and Burst Time for P1: 1 2
Enter Arrival Time and Burst Time for P2: 0 1
Enter Arrival Time and Burst Time for P3: 2 3
Enter Arrival Time and Burst Time for P4: 1 1
Enter Arrival Time and Burst Time for P5: 3 4

Process AT      BT      WT      TAT
P1      0       1       0       1
P2      1       2       1       3
P3      1       1       3       4
P4      2       3       4       7
P5      3       4       7       11
```

2. SJF (Shortest Job First)

SJF selects the process with the smallest burst time for execution. It reduces average waiting time compared to FCFS. It can be preemptive or non-preemptive. However, it may cause starvation for longer processes.

SJF.cpp > main()

```cpp
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int at[n], bt[n], wt[n]={0}, tat[n], ct[n];
    bool done[n] = {false};
    for(int i=0;i<n;i++){
        cout<<"Enter AT and BT for P"<<i+1<<": ";
        cin>>at[i]>>bt[i];
    }

    int completed = 0, time = 0;

    while(completed != n){
        int idx = -1;
        int min_bt = 1e9;

        for(int i=0;i<n;i++){
            if(at[i] <= time && !done[i] && bt[i] < min_bt){
                min_bt = bt[i];
                idx = i;
            }
        }

        if(idx != -1){
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            done[idx] = true;
            completed++;
        } else {
            time++; // CPU idle
        }
    }

    cout<<"\nP\tAT\tBT\tCT\tWT\tTAT\n";
    for(int i=0;i<n;i++){
        cout<<"P"<<i+1<<"\t"<<at[i]<<"\t"<<bt[i]<<"\t"
            <<ct[i]<<"\t"<<wt[i]<<"\t"<<tat[i]<<endl;
    }
}
```

```
Enter number of processes: 5
Enter AT and BT for P1: 0 2
Enter AT and BT for P2: 1
4
Enter AT and BT for P3: 3
4
Enter AT and BT for P4: 1
1
Enter AT and BT for P5: 2
1

P       AT      BT      CT      WT      TAT
P1      0       2       2       0       2
P2      1       4       8       3       7
P3      3       4       12      5       9
P4      1       1       3       1       2
P5      2       1       4       1       2
```

```
PS C:\machine_learning\naive bayes> cd "c:\machine_lear
Enter number of processes: 4
Enter arrival time and burst time for process 1: 0 12
Enter arrival time and burst time for process 2: 2 4
Enter arrival time and burst time for process 3: 3 6
Enter arrival time and burst time for process 4: 8 5

PID     AT      BT      CT      TAT     WT
1       0       12      27      27      15
2       2       4       6       4       0
3       3       6       12      9       3
4       8       5       17      9       4
PS C:\machine_learning\naive bayes>
```

srtf.cpp > main()

```cpp
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter arrival time and burst time for process " << i + 1 << ": ";
        cin >> p[i].arrival >> p[i].burst;
        p[i].pid = i + 1;
        p[i].remaining = p[i].burst;
    }
    int completed = 0, currentTime = 0;
    int shortest = -1;
    int minRemaining = INT_MAX;
    while (completed < n) {
        shortest = -1;
        minRemaining = INT_MAX;
        // Find process with minimum remaining time
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= currentTime && p[i].remaining > 0) {
                if (p[i].remaining < minRemaining) {
                    minRemaining = p[i].remaining;
                    shortest = i;
                }
            }
        }
        if (shortest == -1) {
            currentTime++;
            continue;
        }
        // Execute process for 1 unit
        p[shortest].remaining--;
        currentTime++;
        // If process completes
        if (p[shortest].remaining == 0) {
            completed++;
            p[shortest].completion = currentTime;
            p[shortest].turnaround = p[shortest].completion - p[shortest].arrival;
            p[shortest].waiting = p[shortest].turnaround - p[shortest].burst;
        }
    }
    // Output
    cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
    for (int i = 0; i < n; i++) {
        cout << p[i].pid << "\t"
             << p[i].arrival << "\t"
             << p[i].burst << "\t"
             << p[i].completion << "\t"
             << p[i].turnaround << "\t"
             << p[i].waiting << endl;
    }

    return 0;
}
```

3. Round Robin

Round Robin scheduling assigns each process a fixed time quantum. Processes are executed in a cyclic order. If a process is not completed in its time quantum, it goes back to the queue. It ensures fairness and is widely used in time-sharing systems.

roundRobin.cpp > main()

```cpp
int main(){
    int n, tq;
    cout<<"Enter number of processes: ";
    cin>>n;
    int at[n], bt[n], rem[n], wt[n]={0}, tat[n], ct[n];
    for(int i=0;i<n;i++){
        cout<<"Enter AT and BT for P"<<i+1<<": ";
        cin>>at[i]>>bt[i];
        rem[i]=bt[i];
    }
    cout<<"Enter Time Quantum: ";
    cin>>tq;

    int time = 0, completed = 0;

    while(completed < n){
        bool done = true;

        for(int i=0;i<n;i++){
            if(rem[i] > 0 && at[i] <= time){
                done = false;

                if(rem[i] > tq){
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    ct[i] = time;
                    tat[i] = ct[i] - at[i];
                    wt[i] = tat[i] - bt[i];
                    rem[i] = 0;
                    completed++;
                }
            }
        }

        if(done) time++; // idle
    }

    cout<<"\nP\tAT\tBT\tCT\tWT\tTAT\n";
    for(int i=0;i<n;i++){
        cout<<"P"<<i+1<<"\t"<<at[i]<<"\t"<<bt[i]<<"\t"
            <<ct[i]<<"\t"<<wt[i]<<"\t"<<tat[i]<<endl;
    }
}
```

```
Enter number of processes: 5
Enter AT and BT for P1: 0
2
Enter AT and BT for P2: 1 3
Enter AT and BT for P3: 1 2
Enter AT and BT for P4: 0 3
Enter AT and BT for P5: 3 4
Enter Time Quantum: 2

P       AT      BT      CT      WT      TAT
P1      0       2       2       0       2
P2      1       3       11      7       10
P3      1       2       6       3       5
P4      0       3       12      9       12
P5      3       4       14      7       11
```

4. Priority Scheduling (Preemptive)

Priority Preemptive Scheduling assigns a priority to each process. The CPU always executes the process with the highest priority (lowest number = highest priority). If a new process arrives with higher priority, it preempts the currently running process. This can lead to starvation of low-priority processes.

prioritiyscheduling.cpp > main()

```cpp
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int at[n], bt[n], pr[n], rem[n], ct[n], tat[n], wt[n];
    for(int i=0;i<n;i++){
        cout<<"Enter AT, BT and Priority for P"<<i+1<<": ";
        cin>>at[i]>>bt[i]>>pr[i];
        rem[i]=bt[i];
    }
    int completed = 0, time = 0;

    while(completed < n){
        int idx = -1;
        int highest = 1e9;

        for(int i=0;i<n;i++){
            if(at[i] <= time && rem[i] > 0 && pr[i] < highest){
                highest = pr[i];
                idx = i;
            }
        }

        if(idx != -1){
            rem[idx]--;
            time++;

            if(rem[idx] == 0){
                ct[idx] = time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
                completed++;
            }
        } else {
            time++; // CPU idle
        }
    }

    cout<<"\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n";
    for(int i=0;i<n;i++){
        cout<<"P"<<i+1<<"\t"<<at[i]<<"\t"<<bt[i]<<"\t"<<pr[i]<<"\t"
            <<ct[i]<<"\t"<<wt[i]<<"\t"<<tat[i]<<endl;
    }
}
```

```
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0 2 3
Enter AT, BT and Priority for P2: 1 3 4
Enter AT, BT and Priority for P3: 3 2 6
Enter AT, BT and Priority for P4: 2 3 2
Enter AT, BT and Priority for P5: 4 4 1

P       AT      BT      PR      CT      WT      TAT
P1      0       2       3       2       0       2
P2      1       3       4       12      8       11
P3      3       2       6       14      9       11
P4      2       3       2       9       4       7
P5      4       4       1       8       0       4
```

5. Banker's Algorithm

Banker's Algorithm is used to avoid deadlocks. It checks whether allocating resources keeps the system in a **safe state**. If a safe sequence exists, allocation is allowed; otherwise, the process must wait.

bankersalgo.cpp > main()

```cpp
int main(){
    int n, m;
    cout<<"Enter number of processes: ";
    cin>>n;
    cout<<"Enter number of resources: ";
    cin>>m;
    int alloc[n][m], max[n][m], need[n][m], avail[m];
    cout<<"Enter Allocation Matrix:\n";
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            cin>>alloc[i][j];

    cout<<"Enter Max Matrix:\n";
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            cin>>max[i][j];

    cout<<"Enter Available Resources:\n";
    for(int i=0;i<m;i++)
        cin>>avail[i];

    // Need = Max - Allocation
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];
```

```
    bool finished[n] = {false};
    int safeSeq[n];
    int count = 0;
    while(count < n){
        bool found = false;

        for(int i=0;i<n;i++){
            if(!finished[i]){
                bool possible = true;
                for(int j=0;j<m;j++){
                    if(need[i][j] > avail[j]){
                        possible = false;
                        break;
                    }
                }
                if(possible){
                    for(int j=0;j<m;j++)
                        avail[j] += alloc[i][j];

                    safeSeq[count++] = i;
                    finished[i] = true;
                    found = true;
                } } }
        if(!found){
            cout<<"System is NOT in safe state\n";
            return 0;
        } }
    cout<<"Safe Sequence: ";
    for(int i=0;i<n;i++)
        cout<<"P"<<safeSeq[i]<<" ";
    cout<<endl;
}
```

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
Safe Sequence: P1 P3 P4 P0 P2
```

5. Producer–Consumer (Using Semaphore Logic)

The Producer-Consumer problem involves two processes sharing a buffer. The producer adds data, and the consumer removes it. Synchronization ensures the producer does not add data when the buffer is full, and the consumer does not remove data when empty.

producer.cpp > main()

```cpp
#include <iostream>
using namespace std;

int main(){
    int buffer = 0, max = 5;
    int choice;

    while(true){
        cout<<"\n1. Produce\n2. Consume\n3. Exit\n";
        cin>>choice;

        if(choice == 1){
            if(buffer == max){
                cout<<"Buffer Full!\n";
            } else {
                buffer++;
                cout<<"Produced, Buffer = "<<buffer<<endl;
            }
        }
        else if(choice == 2){
            if(buffer == 0){
                cout<<"Buffer Empty!\n";
            } else {
                buffer--;
                cout<<"Consumed, Buffer = "<<buffer<<endl;
            }
        }
        else break;
    }
}
```

```
1. Produce
2. Consume
3. Exit
1
Produced, Buffer = 1

1. Produce
2. Consume
3. Exit
1
Produced, Buffer = 2

1. Produce
2. Consume
3. Exit
1
Produced, Buffer = 3

1. Produce
2. Consume
3. Exit
1
Produced, Buffer = 4

1. Produce
2. Consume
3. Exit
2
Consumed, Buffer = 3

1. Produce
2. Consume
3. Exit
2
Consumed, Buffer = 2

1. Produce
2. Consume
3. Exit
1
Produced, Buffer = 3

1. Produce
2. Consume
3. Exit
2
Consumed, Buffer = 2

1. Produce
2. Consume
3. Exit
3
```